

Desenvolvimento Back-End



Aula 05

- Exercícios (para treinar)
- Herança
- Polimorfismo

Exercício 01

De acordo com a classe Funcionário abaixo, crie um construtor sem parâmetros (vazio) que deverá atribuir ao cargo o valor “assistente” e um outro construtor que recebe parâmetros correspondentes aos atributos.

Funcionario
- cracha: int - salario: float - cargo: String
Funcionario() Funcionario(c: int, s: float, car: String) //Métodos de acesso calculaAumento(porcentagem: float) calculaAumento(tempo: int)

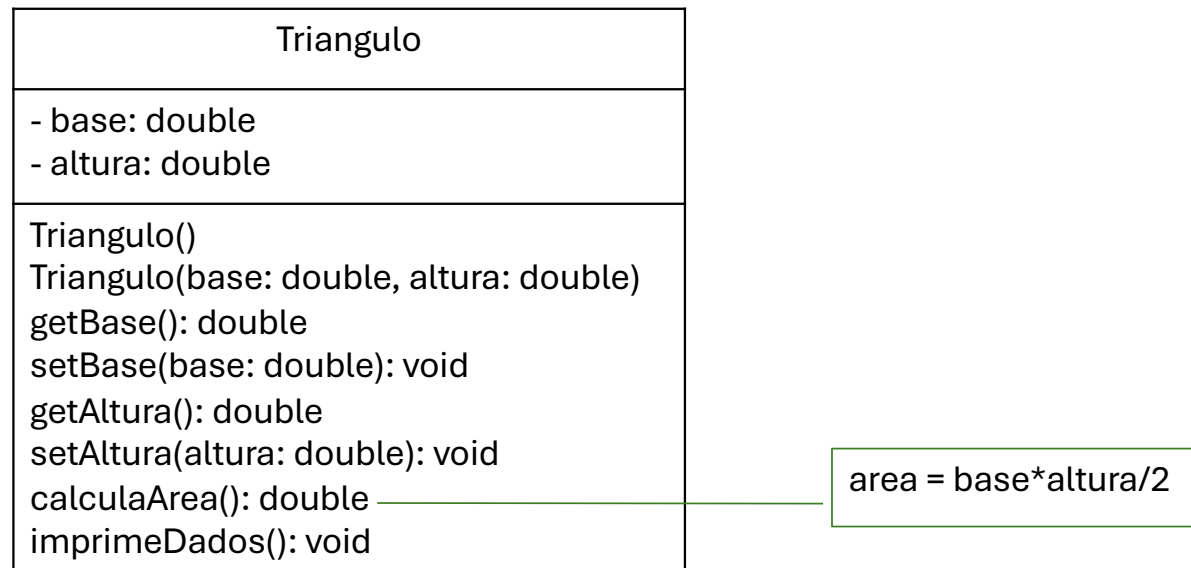
Este método aplica a porcentagem de aumento no salário.

Este método soma R\$150,00 no salário para cada ano trabalhado (recebido por parâmetro).

Exercício 02

Considere o diagrama UML abaixo e altere a classe para acrescentar os modificadores de acesso e os demais métodos necessários.

Em uma classe Java principal (com método main) crie 2 objetos, cada um deve ser instanciado por um construtor diferente. Para o objeto que utiliza o construtor com parâmetros, defina os valores dos atributos. Para o objeto que utiliza o construtor padrão, após a instanciação do mesmo, solicite ao usuário os valores da base e da altura e altere os valores dos atributos utilizando os métodos de acesso. Para ambos, imprima os dados e sua área.



Exercício 03

- Crie uma classe de nome Torneio conforme diagrama e criar classe com void main para instanciar objetos:

Torneio
- nome: string - idade: int
Torneio(nome: string, idade: int) getNome(): string getIdade(): int setNome(n: string): void setIdade(i: int): void verificaCategoria(): string imprimeDados(): void

- a) Crie os sets e gets para cada um dos atributos;
- b) Crie um método imprimirDados que imprime o estado do objeto inclusive sua categoria;
- c) O método verificarCategoria que deverá retornar qual a categoria do atleta baseado na tabela abaixo:

Categoria	Idade
Infantil	5 a 7
Juvenil	8 a 10
Adolescente	11 a 15
Adulto	16 a 30
Sênior	Acima de 30

Exercício 04

- Crie uma classe de nome Vendedor conforme diagrama e criar classe com void main para instanciar objetos:

Vendedor
- vendas: float - salario: float - nome: String - falta: int
Vendedor (v:float, s:float,n:String, f: int) setVendas(v: float): void getVendas(): float setSalario(s: float): void getSalario(): float setNome(n: string): void getNome() : string setFalta(f: int): void getFalta(): int imprimirDados(): void calcularSalario(): void calcularComissao(): float descontoFalta(): float

- a) Crie os sets e gets para cada um dos atributos;
- b) Crie um método imprimirDados que imprime o estado do objeto;
- c) O método calcularComissao deverá retornar o valor da comissão, conforme as regras a seguir:
 - i) venda igual ou acima de 1.000 e menor que 2.000 bônus de 10% sobre o valor das vendas.
 - ii) venda maior ou igual a 2.000 bônus de 15% sobre o valor das vendas.
- d) O método descontoFalta deverá calcular o desconto das faltas conforme o critério:
$$\text{desconto} = (\text{salario} / 30) * \text{falta}$$
- e) O método calcularSalario deverá atender ao critério:
$$\text{salario} = (\text{salario} + \text{comissao} - \text{descontoFalta})$$

Herança

Quando trabalhamos com o paradigma orientado a objetos e começamos a criar nossas primeiras classes, logo entendemos que há uma grande necessidade de compartilhamento de atributos e métodos entre as classes. Isso se dá devido ao fato de uma das características mais importantes da orientação a objetos ser a possibilidade de reutilização de código já escrito anteriormente.

Herança

Como o próprio nome sugere, na orientação a objetos o termo herança se refere a algo herdado, a herança ocorre quando uma classe passa a herdar características **(atributos e métodos)** definidas em uma outra classe, especificada como sendo sua ancestral ou superclasse.

Herança

A técnica da herança possibilita o compartilhamento ou reaproveitamento de recursos definidos anteriormente em uma outra classe. Uma classe derivada herda a estrutura de atributos e métodos de sua classe base, mas pode seletivamente:

- **adicionar novos métodos;**
- **estender a estrutura de dados;**
- **redefinir a implementação de métodos já existentes.**

Herança

Vamos pensar na seguinte situação, precisamos representar dois tipos de veículos: **Carro e Moto**.

Quais seriam as características de Carro?

Herança

Características de Carro:

- **Fabricante;**
- **Modelo;**
- **Cor;**
- **Placa;**
- **Potência em cavalos;**
- **Quantidade de portas;**

Herança

Vamos pensar na seguinte situação, precisamos representar dois tipos de veículos: **Carro e Moto.**

E as características de Moto?

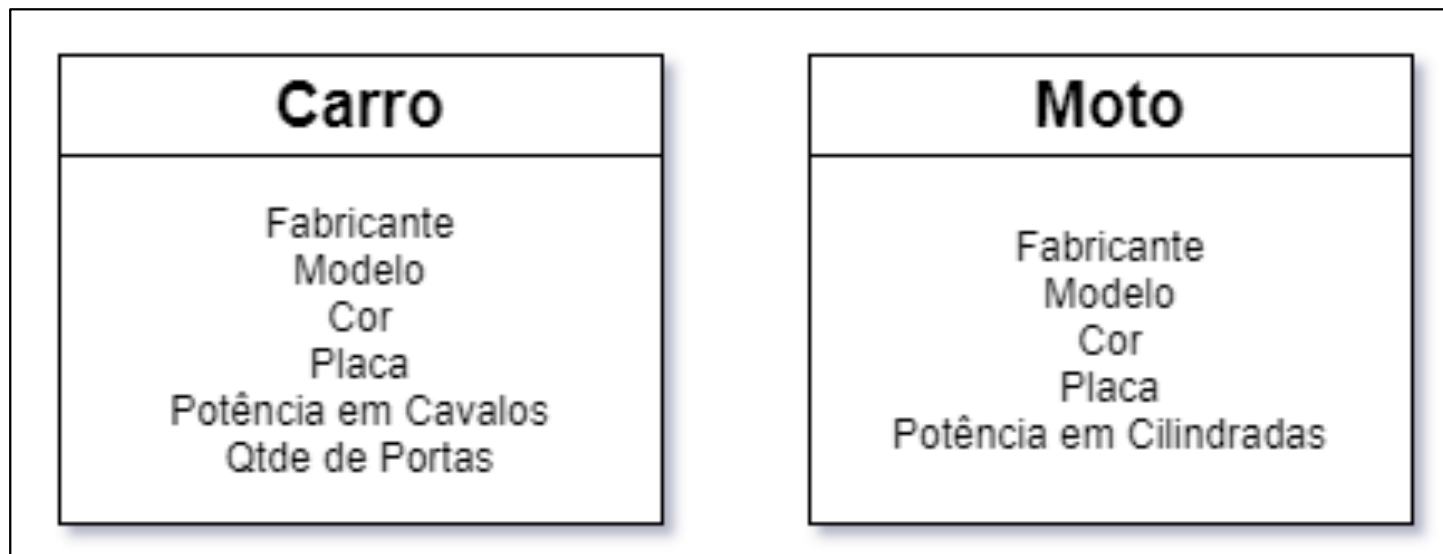
Herança

Características de Moto:

- **Fabricante;**
- **modelo;**
- **Cor;**
- **Placa;**
- **Potência em cilindradas;**

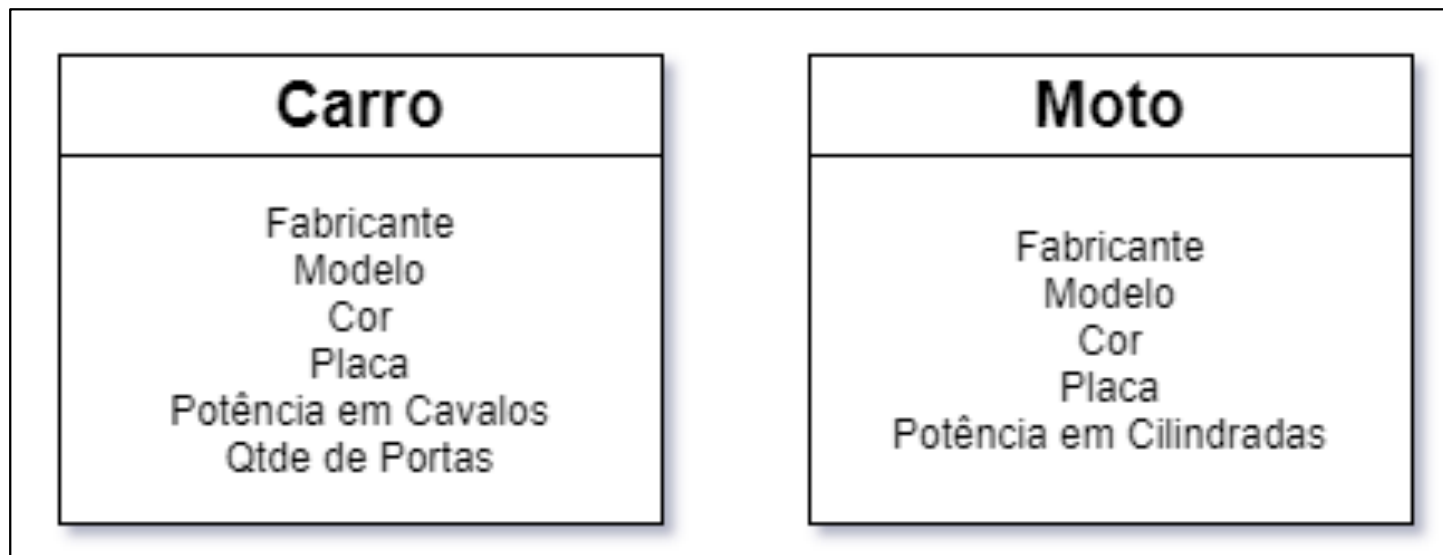
Herança

Classes de Carro e Moto:



Herança

Classes de Carro e Moto:



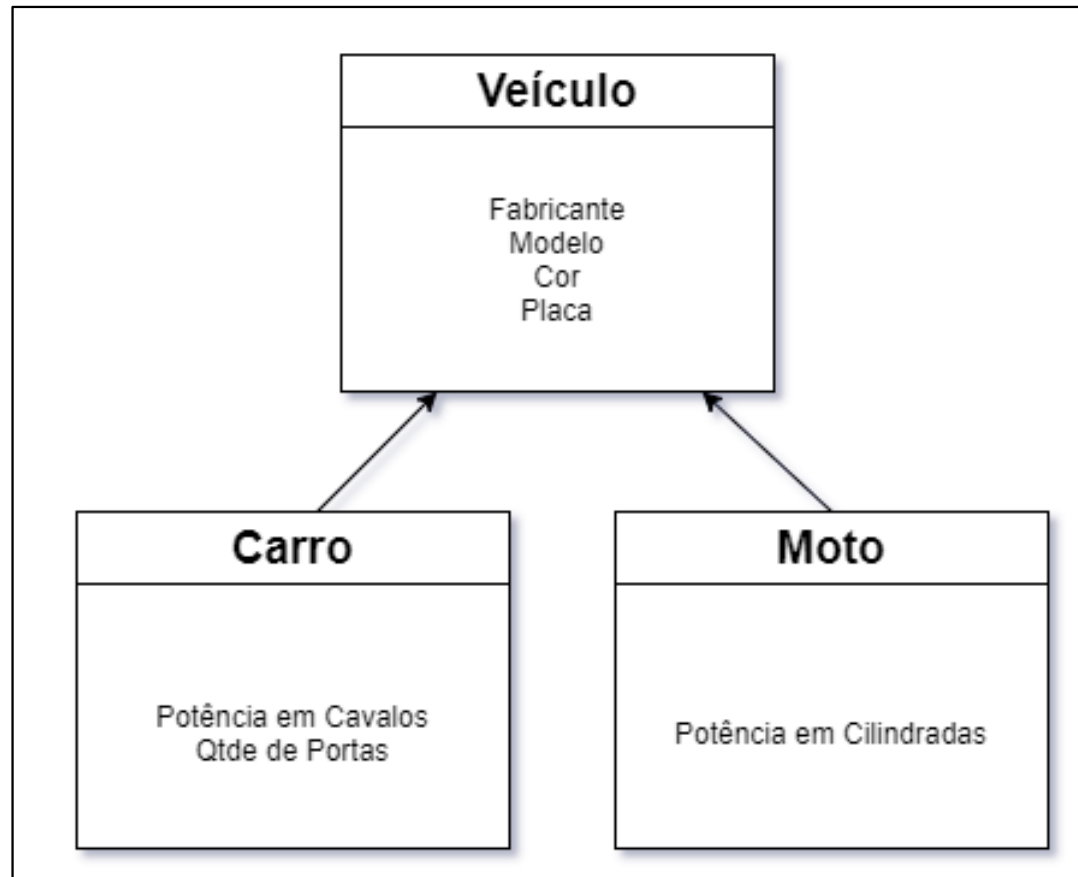
Consegue enxergar algo repetido e que poderia ser reaproveitado?

Herança

Podemos notar que as características (atributos): fabricante, modelo, cor e placa se repetem em ambas as classes, ocasionando **reescrita de código** e indo em um sentido oposto ao que é proposto pela orientação a objetos, pois na orientação a objetos vimos que uma das premissas básicas é a reutilização de código.

Como usar herança nesse caso?

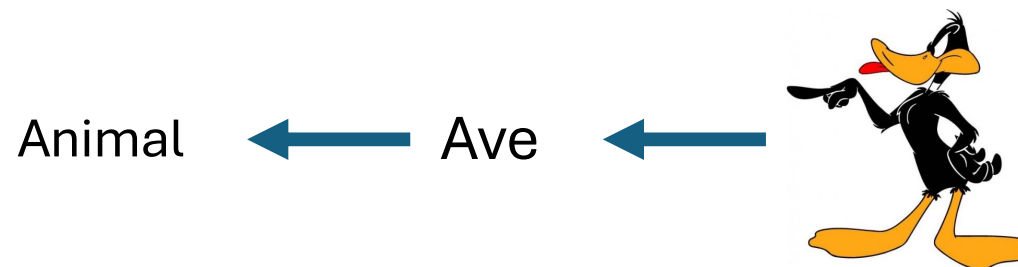
Herança



Herança

A herança pode ser classificada em simples ou múltipla. A herança simples determina que uma classe herdará características de apenas uma superclasse. Já na herança múltipla uma classe herdará características de duas ou mais superclasses (DEITEL, 2010).

Obs.: Apenas as classes filhas acessam informações da mãe, o oposto não ocorre.



Herança

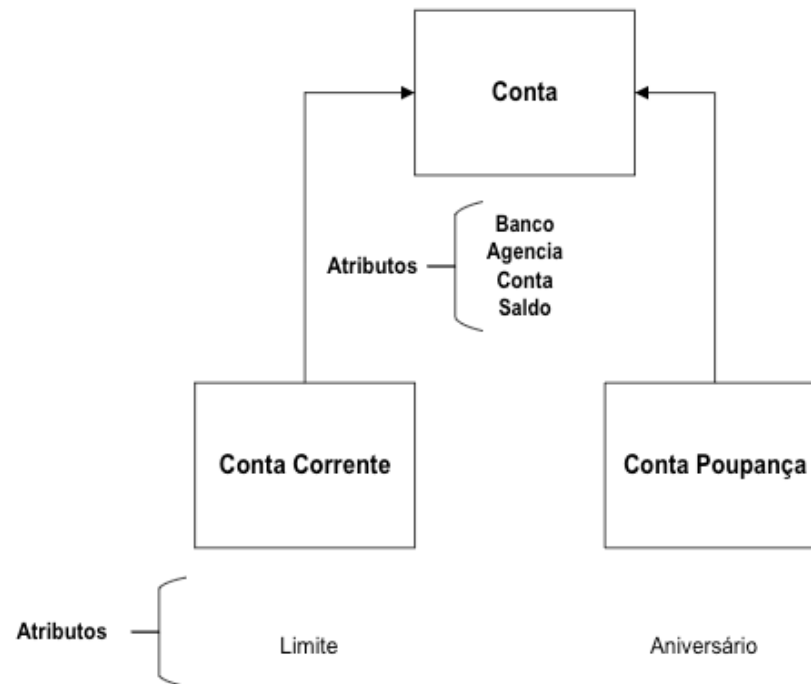


Não ficou repetitivo?

Herança



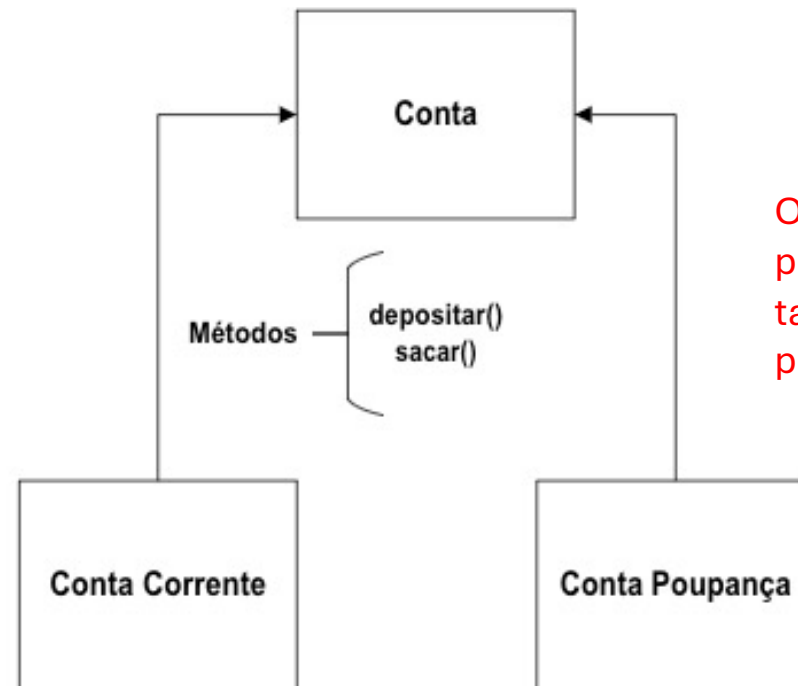
Herança



Sobrescrita de métodos

- A sobrescrita de métodos, ou mais comumente conhecida como override é um procedimento originado na programação à orientação a objetos dentro do conceito de herança.
- Utilizando a sobrescrita de métodos é possível especializar os métodos herdados das classes base, alterando o seu comportamento nas classes derivadas para uma implementação mais específica.
- Podemos dizer que sobrescrever um método é basicamente reescrevê-lo na classe derivada utilizando a mesma assinatura e o mesmo tipo de retorno da classe base, portanto o método deve possuir o mesmo nome, a mesma quantidade e tipos de parâmetros e o mesmo retorno do método herdado.

Sobrescrita de métodos

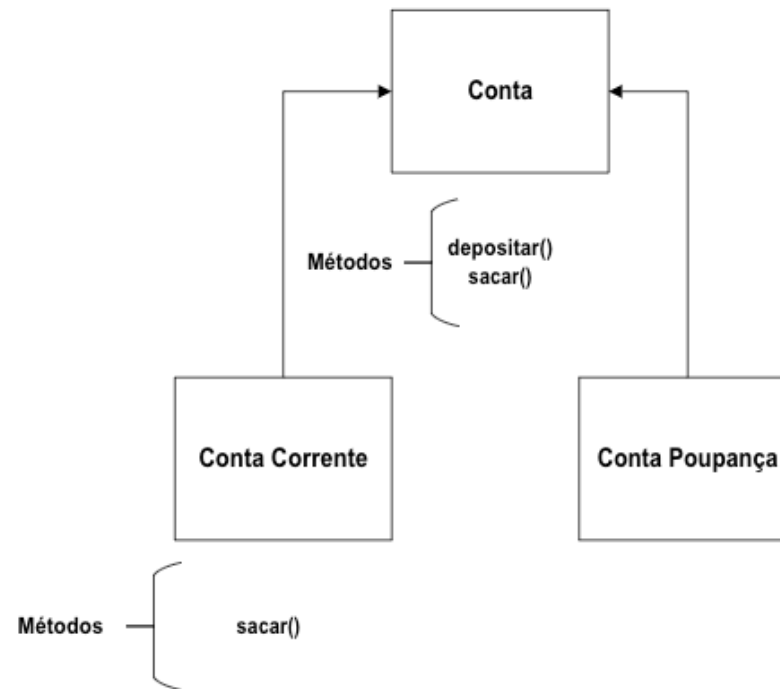


Os métodos depositar e sacar, possuem o mesmo comportamento, tanto para Conta Corrente, quanto para Conta Poupança?

O Sacar não, por causa do limite...

Sobrescrita de métodos

Na classe filha, quando usamos o super podemos invocar os métodos da classe mãe e utiliza-lo da forma que ele foi implementado nela.



Podemos reescrever o método `sacar()` na classe **Conta Corrente**, com isso damos a ele um comportamento correto, considerando o limite no saque.

Polimorfismo

O **polimorfismo indica** a capacidade de abstrair várias **implementações diferentes** em uma única interface.

- **Polimorfismo** é o princípio pelo qual **duas ou mais classes derivadas** de uma mesma **superclasse** podem invocar **métodos** que têm a **mesma identificação** (assinatura) mas **comportamentos distintos**, especializados para cada classe derivada.

Exemplo: Ir para um endereço: As subclasses bicicletas e carro herdam a operação IrParaEndereco da superclasse veículo. No entanto, cada um desses veículos irá executar essa ação de forma diferente da outra.



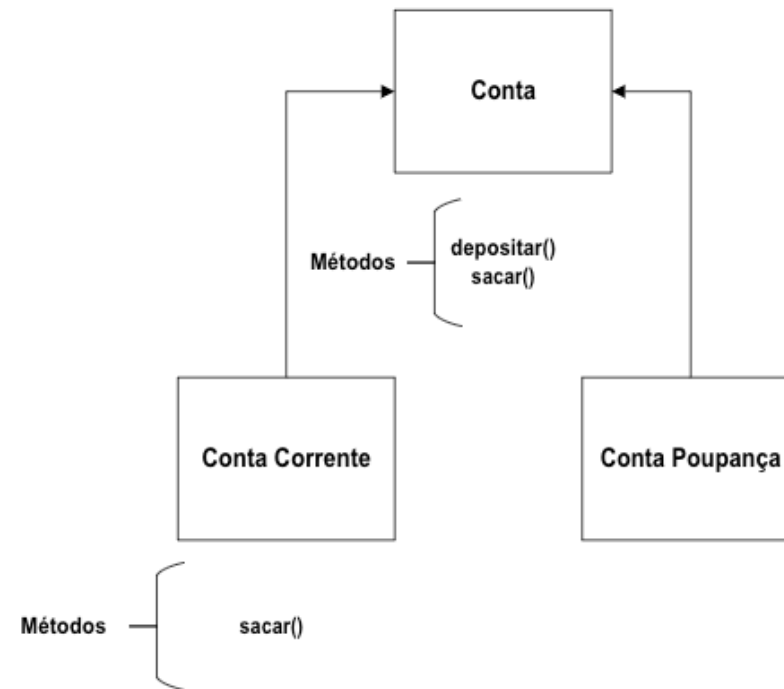
Polimorfismo

- O termo “polimorfismo” é originário do grego e significa “muitas formas” (poli = muitas, morphos = formas). Por exemplo, no jogo de videogame Super Mario, há diferentes personagens que realizam, entre outras, uma mesma ação em comum: correr. O polimorfismo permite que cada personagem (Mario, Luigi e Yoshi) tenha seu próprio modo de correr.
- Podemos definir polimorfismo como o princípio pelo qual as classes derivadas de uma única classe base são capazes de invocar os métodos que, embora apresentem a mesma assinatura, comportam-se de forma específica para cada uma das classes derivadas (FELIX, 2016).
- Utilizando o polimorfismo, os mesmos atributos e objetos podem ser usados em objetos diferentes, porém com implementações lógicas distintas.

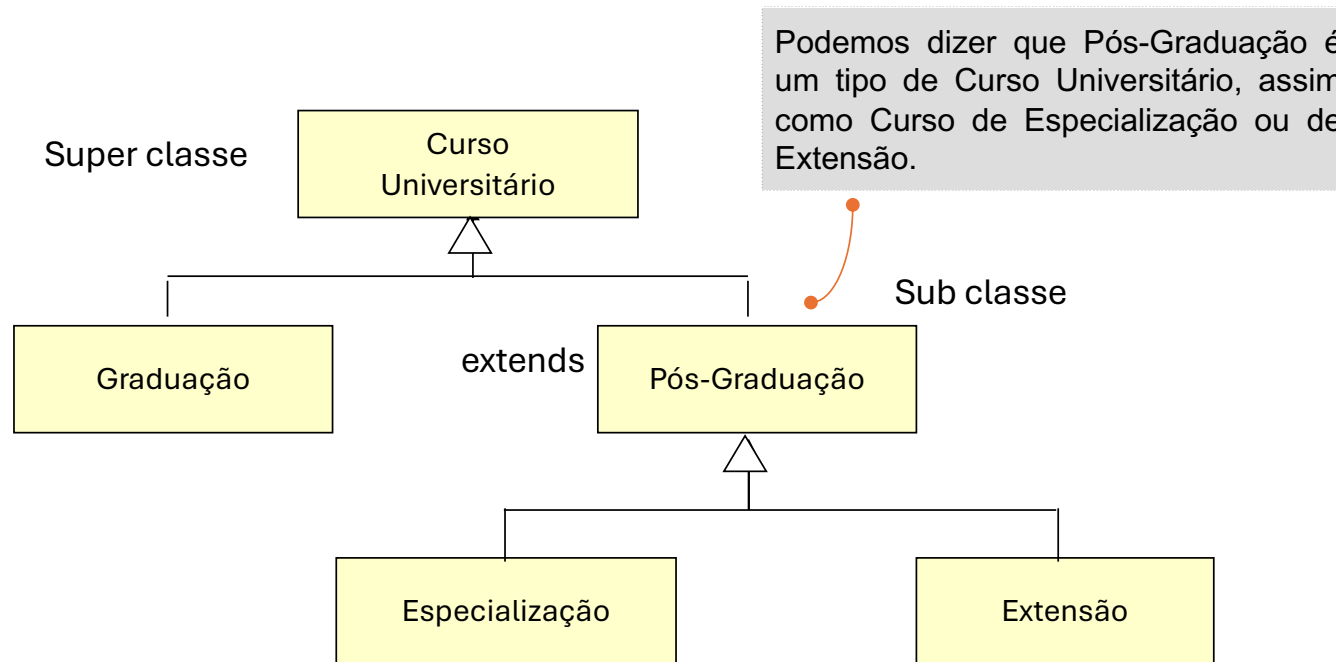
Polimorfismo

O método sacar() tem um comportamento na classe Conta e um comportamento diferente na classe Conta Corrente...

Isso é Polimorfismo...poli(muitas) morfos (formas)...



Herança



Herança em Java

- Para dizer que uma classe implementa herança, utilizamos a palavra-chave *extends* logo após a declaração da mesma:

```
public class Filho extends Pai{ }
```

- Se uma classe não contém a declaração *extends*, automaticamente “herda” as características da classe **Object**
- Alguns métodos da classe **Object** são:
 - `toString() : String`
 - `equals( Object ) : boolean`

Herança – Modificadores de Acesso

protected: podem ser acessados pelos membros da subclasse

private: só podem ser acessados pela própria classe

```
public class Pai {  
    public int a;  
    protected int b;  
    private int c;  
  
    public Pai() {  
        a=10;  
        b=24;  
        c=66;  
    }  
}
```

Pai.java

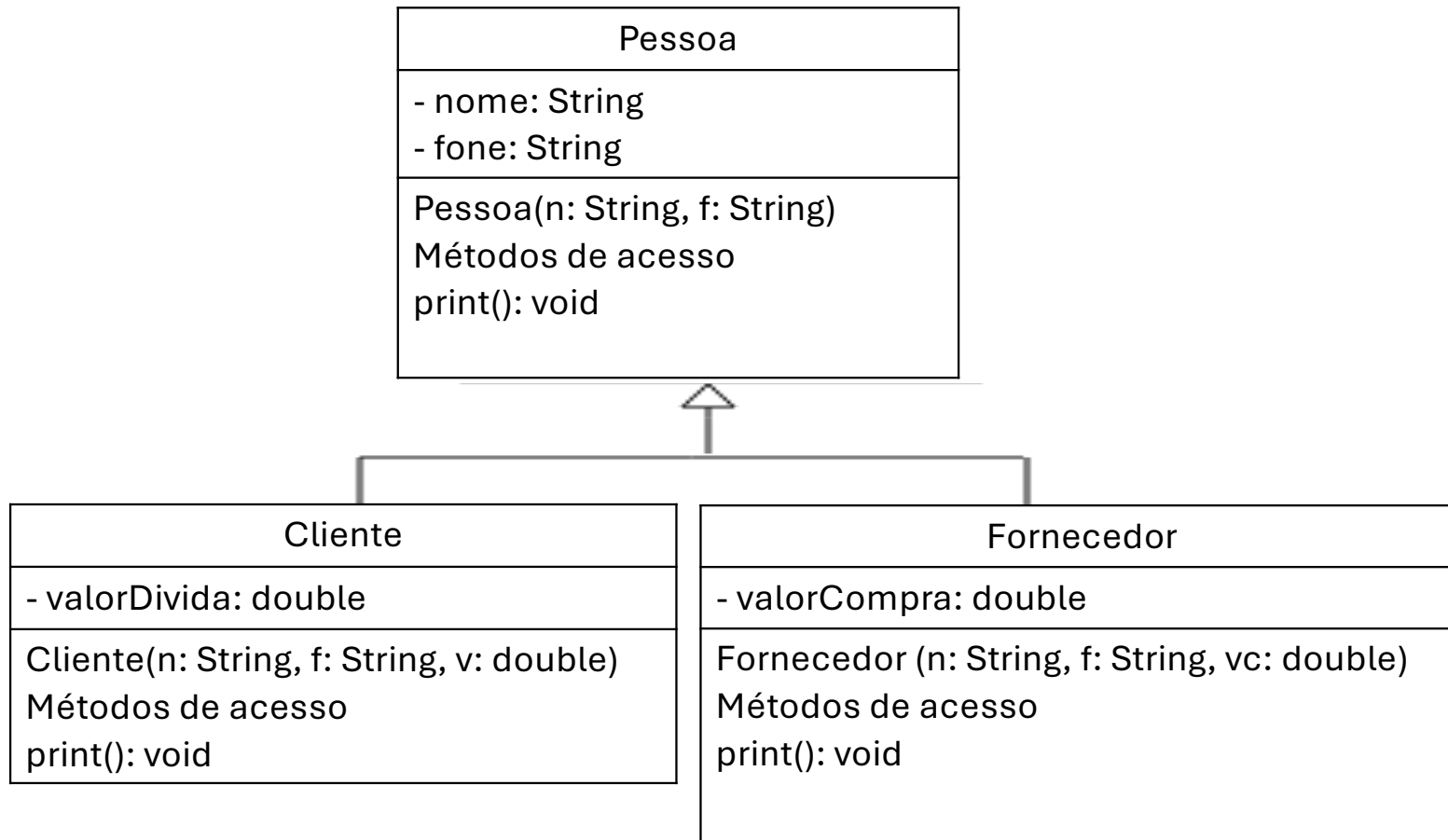
```
public class Filho extends Pai{  
  
    public void imprimeFilho() {  
        System.out.println("Valor de a: " + a);  
        System.out.println("Valor de b: " + b);  
        System.out.println("Valor de c: " + c);  
    }  
}
```

Filho.java

```
public class TestaFilho {  
  
    public static void main(String[] args) {  
        Filho f=new Filho();  
        f.imprimeFilho();  
    }  
}
```

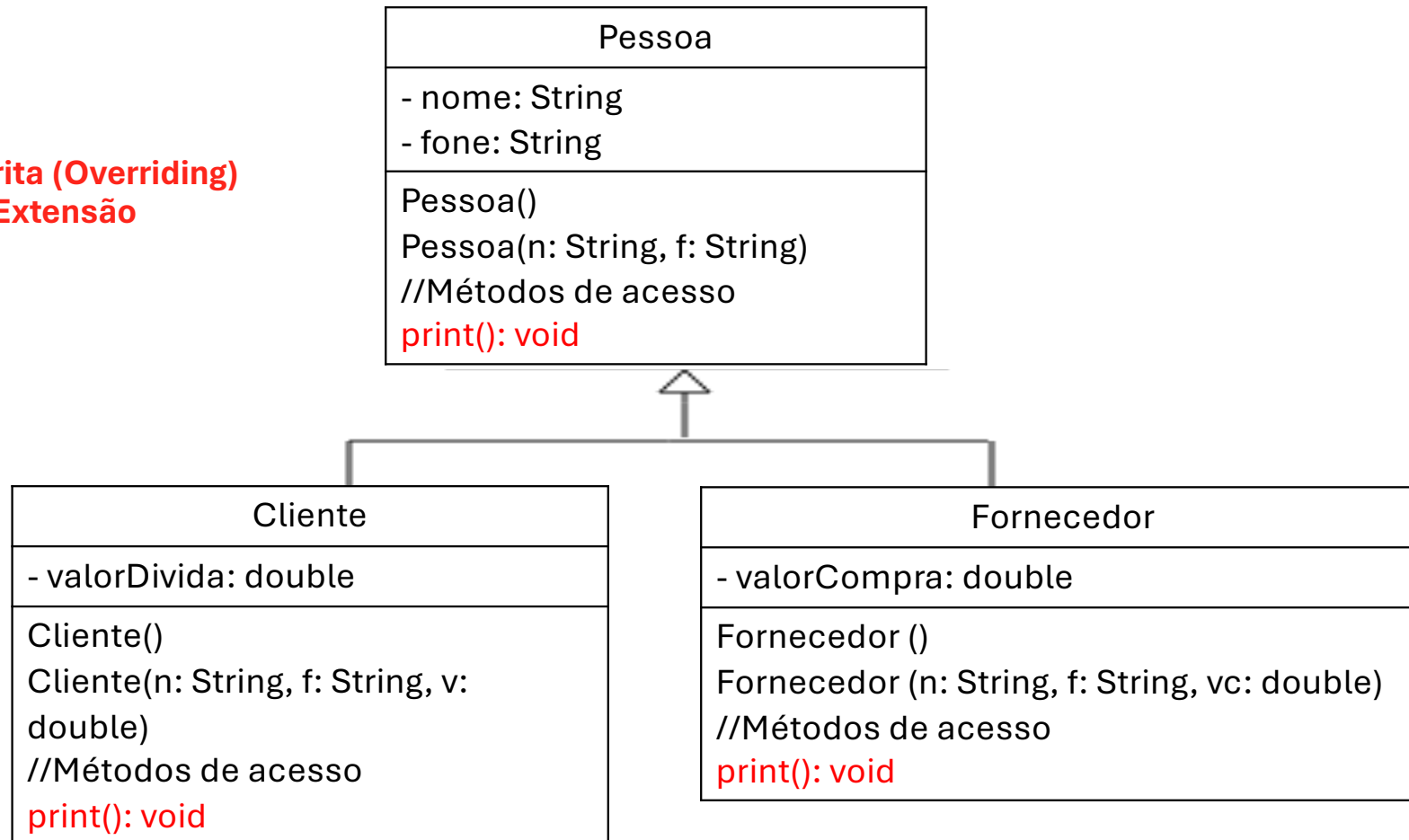
TestaFilho.java

Exemplo 1



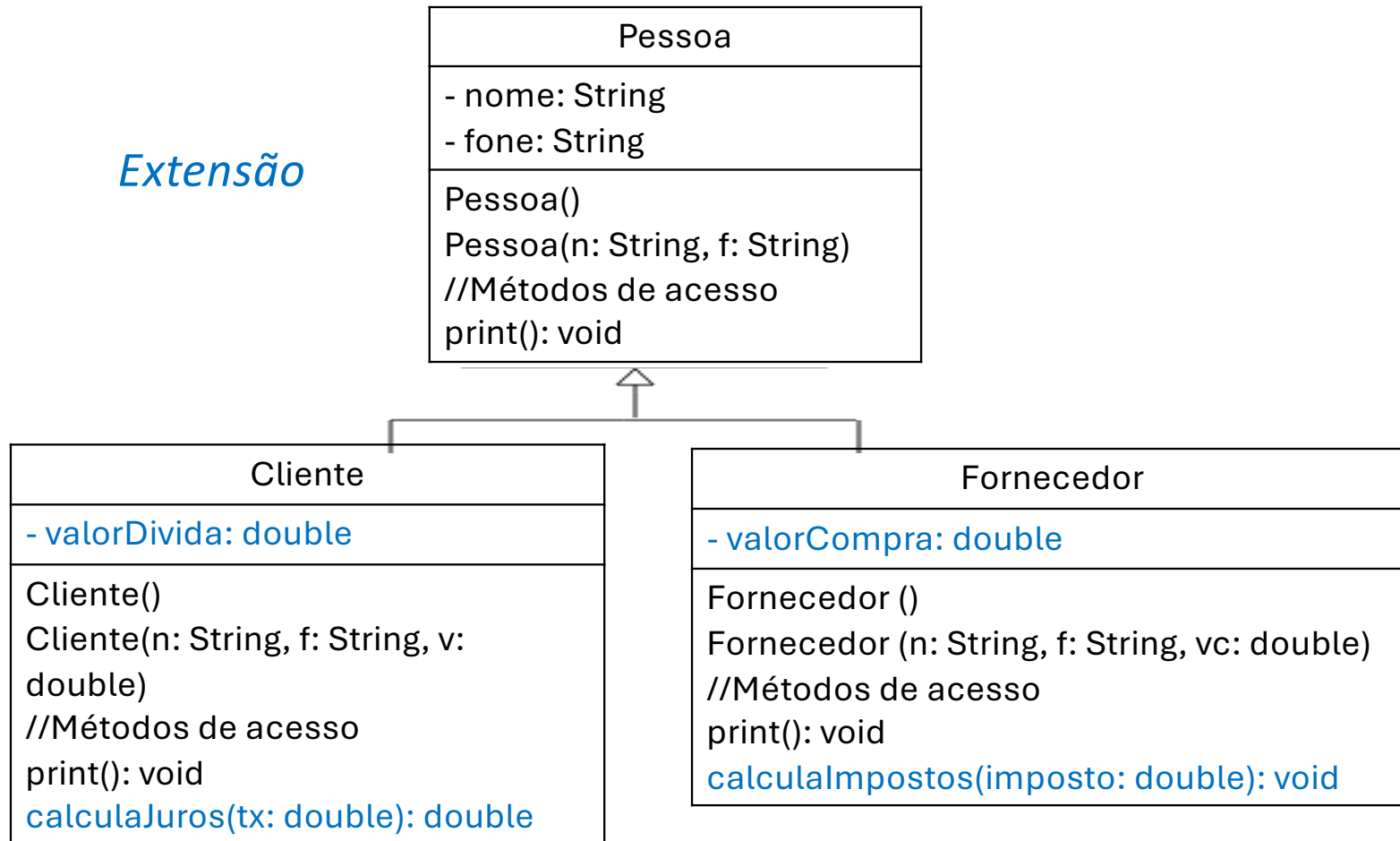
Exemplo 1

Sobrescrita (Overriding)
e Extensão



Exemplo 1- complemento

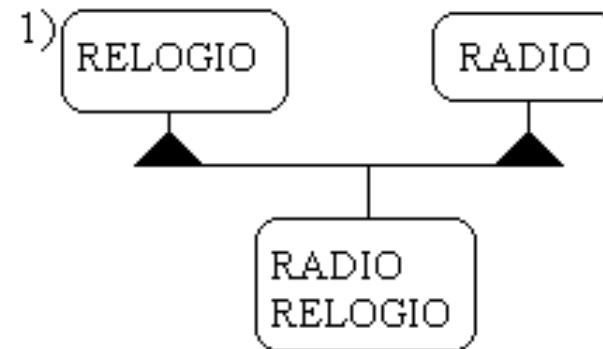
Extensão



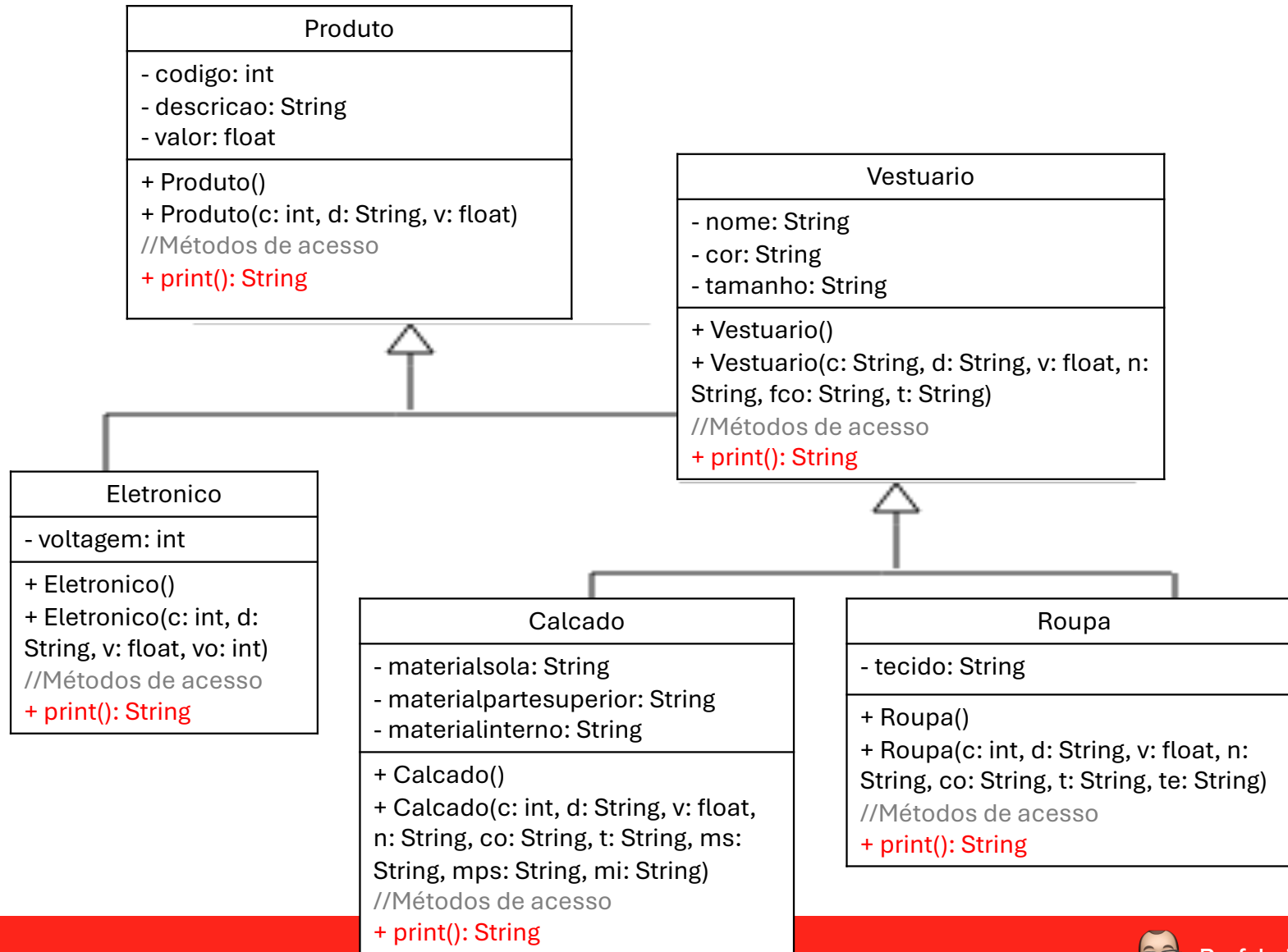
Herança Múltipla

Algumas linguagens OO permitem fazer herança múltipla como o C++, o que significa que uma subclasse pode herdar características de duas ou mais classes. Isso permite que uma classe agrupe atributos e métodos de várias classes.

Java não tem herança múltipla



Exemplo 2



Já sei tudo?

- Só isso? Já sei tudo que diz respeito à orientação a objetos?
- Não, ainda há um longo caminho pela frente, com muitos conceitos importantes ;
- Agora que tudo começa a ficar realmente divertido! 😊 😊 (é sério!)





That's all Folks!